

BAR-K9: Behavioral Autonomous Robotic - K9

Aditya Rampal
Robotics & Autonomous Systems
Boston University
Boston, MA, USA
rampal11@bu.edu

Abin Binoy George
Electrical and Computer Engineering
Boston University
Boston, MA, USA
abg309@bu.edu

Abstract—The primary goal of robotics is to develop autonomous systems that assist humans in various tasks. For these systems to be effective, robots must adapt to unfamiliar environments and respond to unexpected events almost instantaneously. This challenge represents a significant and highly sought-after area of research known as system identification. Typically, model-based control methods are ineffective due to complex, non-linear, and non-deterministic relationships. In this study, we propose using *Reinforcement Learning (RL)* to guide a quadruped robot navigating uneven and rough terrains. We tested the RL model on a MuJoCo simulation and created an XML file of "PATRIQ," a quadruped developed by a previous EC518 student, to assess its performance under various conditions. This evaluation helps highlight the model's ability to adapt to new terrain and the viability of the quadruped's design for real-world applications. If time permitted, we planned to incorporate an extra layer of controls, enabling the quadruped to function primarily as a "guide dog," tasked with following a specific human. However, this was not implemented in the current scope. **Index Terms**—Robotics, Reinforcement Learning, Quadruped Robots, PATRIQ, Terrain Adaptation, Guide Dog

INTRODUCTION

The rapid evolution of robotics technology has ushered in a new era of autonomous systems designed to assist humans across a myriad of tasks. A critical aspect of this field lies in the ability of robots to operate effectively in dynamic and unpredictable environments. This requirement poses a significant challenge, as traditional model-based control methods often fall short when faced with the complexities of non-linear and nondeterministic interactions found in real-world scenarios. Consequently, there has been a growing interest in innovative approaches to enhance robotic adaptability and responsiveness.

This study explores the application of Reinforcement Learning (RL) as a means to empower a quadruped robot (figure 1(a)) in navigating rough and uneven terrains. By leveraging RL, we aim to create a system that can learn from its interactions with the environment, enabling it to make real-time decisions and adjustments. Moreover, we intend to augment the quadruped's capabilities by incorporating an additional layer of control, transforming it into a "guide dog" that follows a specific human companion. This dual focus not only underscores the versatility of RL in robotic applications but also highlights the potential for practical use in assisting individuals in complex environments.

Another aim of this project was to conduct a comparative analysis between a pre-built quadruped (figure 1(a)) and a

newly designed quadruped "PATRIQ" developed by a previous EC518 student (figure 1(b)). This comparison aims to evaluate the performance differences in adapting to various terrains, contributing valuable insights to the ongoing discourse in robotics research. Through this study, we hope to advance our understanding of how adaptive learning techniques can enhance the functionality of autonomous robotic systems in real-world applications.

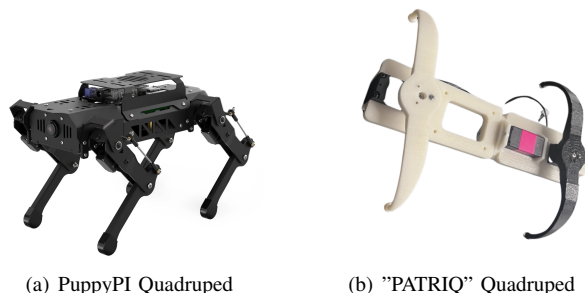


Fig. 1. Quadrupeds used for the Project

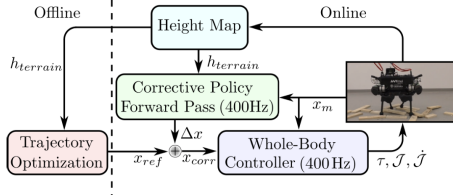
MOTIVATION

In recent years, there has been an increasing demand for adaptable and agile robotics applications, such as quadrupeds, that demonstrate improved locomotion in complex terrains. The current state-of-the-art methods aim to address the challenges of navigating uneven terrain while ensuring stability and efficiency at every step of the system's operation. In this context, reinforcement learning (RL) emerges as one of the promising methods to overcome these limitations by enabling robots to learn and adapt through interactions with their surroundings.

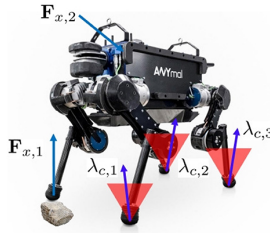
Previous works, such as "*Real-Time Trajectory Adaptation for Quadrupedal Locomotion using Deep Reinforcement Learning*" [1], have explored solutions to this problem. The study in [1] utilizes a hybrid approach that combines trajectory optimization and a corrective RL policy. As shown in Fig. 2(b), this framework enables the quadruped to adapt its movement dynamically, ensuring stable traversal through challenging terrains.

Our work builds upon this framework by incorporating RL to help the quadruped traverse various types of complex terrain. Specifically, the approach helps guide the "ANYmal"

quadruped (Fig. 2(b) while maintaining stability and trajectory feasibility. Understanding how the authors in [1] effectively combine trajectory optimization and corrective policies is essential to advancing the performance and adaptability of legged robots in real-world applications.



(a) Trajectory Optimization Framework from [1]



(b) "ANYmal" Quadruped

STATE-OF-THE ART

Legged robots excel in navigating complex terrains where conventional systems fail. The authors in [1] ensure the stability of the 'ANYmal' as seen in Fig. 2(b) quadruped using Trajectory Optimization, Whole-Body Controllers, and Height Maps to navigate challenging environments effectively.

Trajectory Optimization

This framework is used to compute optimal trajectories for legged systems, ensuring stability and feasibility as discussed in [1] under dynamic and kinematic constraints. The approach presented from [2] as seen in Fig. 2(c) optimizes the robot's motion trajectory by combining dynamic modeling and kinematic constraints. The main objective is to find feasible trajectories for the center of mass (CoM) of the robot and the foot placements while respecting physical and terrain-based constraints. The kinematic model in Fig. 2(d) of the "ANYmal" robot from Fig. 2(b) shows that the quadruped after employing the trajectory optimization framework from [1] [2] can find the optimal trajectory along a particular terrain and can be optimized over the dynamic model and the kinematic constraints. This can be seen as follows:

The optimization aims to find:

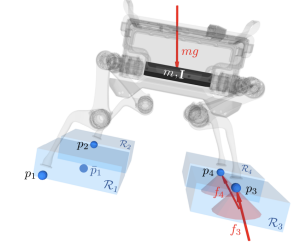
- **Center of Mass (CoM) trajectory:** $\mathbf{r}(t) \in \mathbb{R}^3$,
- **Base Euler angles:** $\boldsymbol{\theta}(t) \in \mathbb{R}^3$,
- Foot positions $\mathbf{p}_i(t)$ and forces $\mathbf{f}_i(t)$ for each foot i ,

```

find  $\mathbf{r}(t) \in \mathbb{R}^3$  (CoM linear position)
 $\boldsymbol{\theta}(t) \in \mathbb{R}^3$  (base euler angles)
for every foot  $i$ :
 $\Delta T_{i,1}, \dots, \Delta T_{i,2n_{s,i}} \in \mathbb{R}$  (phase durations)
 $\mathbf{p}_i(t, \Delta T_{i,1}, \dots) \in \mathbb{R}^3$  (foot position)
 $\mathbf{f}_i(t, \Delta T_{i,1}, \dots) \in \mathbb{R}^3$  (force at foot)
s.t.  $[\mathbf{r}, \boldsymbol{\theta}](t=0) = [\mathbf{r}_0, \boldsymbol{\theta}_0]$  (initial state)
 $\mathbf{r}(t=T) = \mathbf{r}_g$  (desired goal)
 $[\ddot{\mathbf{r}}, \ddot{\boldsymbol{\omega}}]^T = \mathbf{f}_d(\mathbf{r}, \mathbf{p}_1, \dots, \mathbf{f}_1, \dots)$  (dynamic model)
for every foot  $i$ :
 $\mathbf{p}_i(t) \in \mathcal{R}_i(\mathbf{r}, \boldsymbol{\theta})$  (kinematic model)
if foot  $i$  in contact:
 $\mathbf{p}_i(t \in \mathcal{C}_i) = \mathbf{0}$  (no slip)
 $\mathbf{p}_i^z(t \in \mathcal{C}_i) = h_{\text{terrain}}(\mathbf{p}_i^{xy})$  (terrain height)
 $\mathbf{f}_i(t \in \mathcal{C}_i) \cdot \mathbf{n}(\mathbf{p}_i^{xy}) \geq 0$  (pushing force)
 $\mathbf{f}_i(t \in \mathcal{C}_i) \in \mathcal{F}(\mu, \mathbf{n}, \mathbf{p}_i^{xy})$  (friction cone)
if foot  $i$  in air:
 $\mathbf{f}_i(t \notin \mathcal{C}_i) = \mathbf{0}$  (no force in air)
 $\sum_{j=1}^{2n_{s,i}} \Delta T_{i,j} = T$  (total duration)

```

(c) TO framework presented in [2]



(d) Kinematic Model of 2(b) [2]

Fig. 2. "Gait and Trajectory Optimization for Legged Systems through Phase-based End-Effector Parametrization" [2]

subject to the following constraints:

$$[\mathbf{r}, \boldsymbol{\theta}](t=0) = [\mathbf{r}_0, \boldsymbol{\theta}_0], \quad (\text{initial state}) \quad (1)$$

$$\mathbf{r}(t=T) = \mathbf{r}_g, \quad (\text{goal state}) \quad (2)$$

$$[\ddot{\mathbf{r}}, \ddot{\boldsymbol{\omega}}]^T = \mathbf{f}_d(\mathbf{r}, \mathbf{p}_1, \dots, \mathbf{f}_1, \dots). \quad (\text{dynamic model}) \quad (3)$$

Kinematic and Contact Constraints

For each foot i , the following constraints apply:

• Kinematic constraints:

$$\mathbf{p}_i(t) \in \mathcal{R}_i(\mathbf{r}, \boldsymbol{\theta}).$$

• Contact constraints (foot in contact):

$$\dot{\mathbf{p}}_i = \mathbf{0} \quad (\text{no slip}), \quad (4)$$

$$\mathbf{p}_i^z = h_{\text{terrain}}(\mathbf{p}_i^{xy}) \quad (\text{terrain height}), \quad (5)$$

$$\mathbf{f}_i \cdot \mathbf{n} \geq 0 \quad (\text{pushing force}), \quad (6)$$

$$\mathbf{f}_i \in \mathcal{F}(\mu, \mathbf{n}, \mathbf{p}_i^{xy}) \quad (\text{friction cone}). \quad (7)$$

• No forces in air (foot not in contact):

$$\mathbf{f}_i = \mathbf{0}.$$

The phase durations $\Delta T_{i,j}$ are optimized to satisfy:

$$\sum_{j=1}^{2n_{s,i}} \Delta T_{i,j} = T \quad (\text{total duration}).$$

The results demonstrate the optimized **CoM trajectory** and **foot placements**, enabling stable traversal of uneven terrain. Visualization of the force vectors and optimized trajectories highlights the robustness of this approach.

Whole Body Controller

Once the quadruped can stabilize its trajectory and body while traversing complex terrain, it needs to track the motion plan set points based in [3] that incorporates an approach known as "hierarchical optimization" to compute all the necessary feed-forward joint torques. Whilst, the Trajectory Optimization [TO] from [2] only computes constant forces to generate the trajectories, the work presented in [1] establishes that the Whole Body Controller [WBC] is responsible for

taking the feet and base trajectories as inputs and recomputing the contact forces utilizing the full dynamics model of the robot. WBC consists of a torque-based control framework and enables unified control of motion and force which is extremely beneficial to our project.

Height Map

Figure 3 shows an example of a classic height map used in simulation to observe the current terrain. This information is typically fed into the RL encoder to extract the main features of the height map. The extracted feature set is then used in the neural network of the RL policy for training [1] [2].

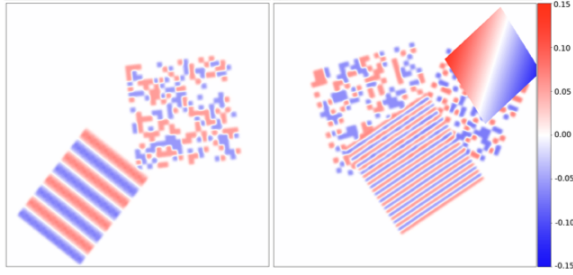


Fig. 3. The corresponding 16-bit PNG images represent the height maps. The color bar represents the terrain elevation in m [1].

METHOD

"The RL framework used in this project builds upon the approach presented in [1], where a combination of Reinforcement Learning [RL] algorithms such as Soft Actor-Critic (SAC) and Deep Q-Learning (DQL) is incorporated alongside a reward function to train a policy for stable quadrupedal locomotion. The framework applies similar logic as [1] as the policy generates actions, as torque commands that are based on observations, and finally are validated through *MuJoCo* simulator before deploying the final policy on the *PuppyPI* and *PATRIQ* quadrupeds used in this project as seen in Fig. 1

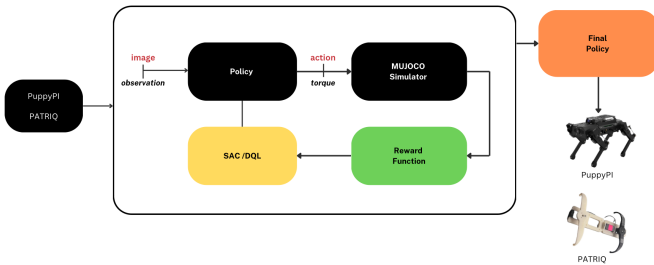


Fig. 4. RL Training Framework for BAR-K9 project

The components of the RL Training Framework are as follows:

Policy:

The decision-making module that maps observations as images to actions torque commands to control the quadruped.

MuJoCo Simulator:

A High Fidelity physics simulator that applies the torques and provides new observations and environment states (feedback).

Reward Function:

Evaluates the performance of the policy by assigning rewards based on the desired behavior, such as stability, energy efficiency, or trajectory tracking. The reward function comprises as follows:

- (1). Reward for forward progress
- (2). Penalty for energy usage
- (3). Reward for Stability
- (4). Reward for terrain adaptability
- (5). Total Reward = fwd_reward
+ energy_penalty
+ orientation_penalty
+ terrain_adapt_reward

SAC + DQL:

Two Reinforcement Learning algorithms that are used for training *PATRIQ* quadruped (Fig. 1(b))

Soft Actor-Critic (SAC): Optimizes the policy for continuous actions with exploration and stability

Deep Q-Learning (DQL): Enhances policy training for discrete decision-making tasks.

SIMULATOR

In order to develop a successful reinforcement learning (RL) framework for *PATRIQ*, we need to establish a simulation framework that can accurately model the robot across various environments. We evaluated different simulators, including *Gazebo*, *PyBullet*, and *MuJoCo* [4]. After conducting a thorough analysis, we chose *MuJoCo* due to its advanced and precise physics engine [4]. However, this decision also required us to manually create *MuJoCo* specific XML files for *PATRIQ*. Additionally, we had to design multiple environments in which to run the RL framework, ensuring that the RL algorithm could effectively learn a policy enabling *PATRIQ* to navigate diverse terrains.

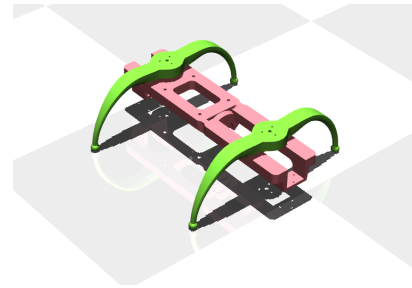


Fig. 5. *PATRIQ* in *MuJoCo*

REINFORCEMENT LEARNING ALGORITHMS

Deep Q-Learning (DQL)

In this section, we will discuss the Reinforcement Learning algorithms that we have implemented in our project. By using



Fig. 6. PATRIQ in *MuJoCo* with a height field.

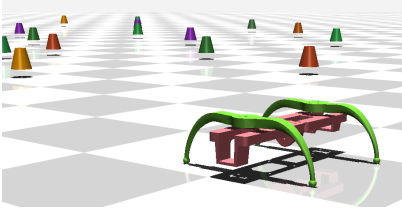


Fig. 7. PATRIQ in *MuJoCo* with obstacles.

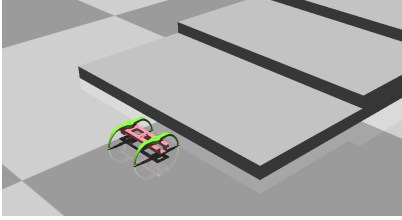


Fig. 8. PATRIQ in *MuJoCo* with stairs.

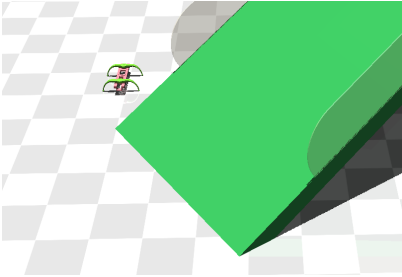


Fig. 9. PATRIQ in *MuJoCo* with a slope.

two different algorithms, we can compare the performance of the robot between the two algorithms.

Deep Q-Learning (DQL)

Deep Q-Learning is a model-free reinforcement learning algorithm to learn a policy telling an agent what action to take under what circumstances. It uses a deep neural network to approximate the Q-value function. The Q-value function is a function that takes the state and action as input and returns the expected reward. The Q-value function is updated using the Bellman equation. The Bellman equation is as follows:

$$Q(s, a) = r + \gamma \max_{a'} Q(s', a')$$

where, s' = next state

a' = next action

r = reward

γ = discount factor

The main idea behind the Bellman equation to choose the appropriate action in a given state is to maximize the expected reward.

Additionally, in the Deep Q-Learning algorithm, we use 2 neural networks: evaluation network and target network. The evaluation network takes the state as input and returns the Q-value for the predicted action. The target network is simply a copy of the evaluation network that is updated every set number of iterations. The target network is used to calculate the the maximum Q-value for the given state. We compare the Q-value of the evaluation network with the Q-value of the target network to calculate the loss. The loss is then backpropagated to update the weights of the evaluation network. The goal of using an eval and target network is to stabilize the training process.

Moreover, we also use experience replay in the Deep Q-Learning algorithm. Experience replay is a technique that stores the agent's experiences in a replay buffer, which consists of the state, action, reward, next state, and done. The agent samples a batch of experiences from the replay buffer and uses them to update the Q-value function. This technique helps to break the correlation between the experiences and stabilize the training process. In other words, it helps to prevent the agent from overfitting to the most recent experiences.

The general ideal of the DQL algorithm is to learn the optimal policy by maximizing the expected reward. The agent learns the optimal policy by exploring the environment and updating the Q-value function using a deep neural network. The agent learns the optimal policy by taking the action that maximizes the expected reward in a given state. Additionally, the agent uses experience replay to stabilize the training process and prevent overfitting to the most recent experiences. Figure 10 below shows the general architecture of the DQL algorithm.

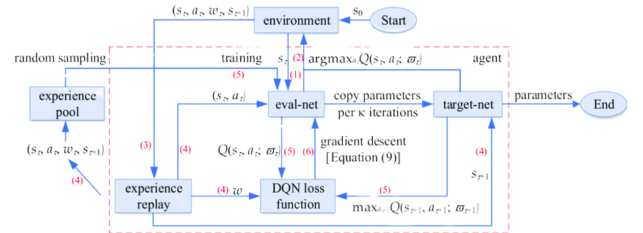


Fig. 10. Deep Q-Learning Architecture

Soft Actor Critic (SAC)

Additionally, we also implemented the Soft Actor Critic (SAC) algorithm in our project. The SAC algorithm is a model-free reinforcement learning algorithm that learns a stochastic policy. At its core, the SAC algorithm is somewhat similar to the DQL algorithm. The main difference between the two algorithms is that the SAC algorithm [5] emphasizes the entropy maximization of the policy. In other words, the SAC algorithm tries to maximize exploration in its policy by randomly selecting actions that have not been explored before for a given state. While the SAC algorithm tries to maximize the entropy of the policy, it also tries to maximize the expected reward.

Figure 11 below shows the general architecture of the SAC algorithm.

Actor Network: The actor network takes the state as input and returns the mean and standard deviation of each possible action. The mean and standard deviation are used to sample the action from a normal distribution and chooses the action which is predicted to yield the greatest reward. The actor network is updated using the reparameterization trick. The reparameterization trick is a technique that samples the action from a normal distribution by adding noise to the mean and multiplying the standard deviation. The actor network is updated by maximizing the expected reward and entropy of the policy.

Critic Network: The critic network takes the state and action as input and returns the Q-value for the predicted action. From figure 11, we can see that there are two critic networks. One critic network is used to calculate the soft Q-value, which is the expected return for the given state and action including the entropy bonus from choosing that particular action. The standard deviation of the action is used to calculate the entropy bonus. Actions that have a high standard deviation means that the agent is uncertain about the action. The entropy bonus is used to encourage the agent to explore the environment and maximize the entropy of the policy. The other critic network is used to calculate the soft V value, which is the expected return for the given state. This value is essentially the weighted average of all the possible actions weighted by the probability of selecting that action. The main difference between the soft Q-value and soft V value can be thought of as the soft Q-value is the expected value for taking a particular action in a given state, while the soft V value is the expected value for being in a given state. The Q-values and the V-values are used to calculate the loss. The loss is then backpropagated to update the weights of the actor and critic networks. It may be possible to just use one critic network, but using two critic networks helps to stabilize the training process. In other words, it helps to prevent the agent from overfitting to the most recent experiences.

Replay Buffer: The replay buffer stores the agent's experiences, which consists of the state, action, reward, next state, and done. The agent samples a batch of experiences from the replay buffer and uses them to update the actor and

critic networks. The replay buffer helps to break the correlation between the experiences and stabilize the training process.

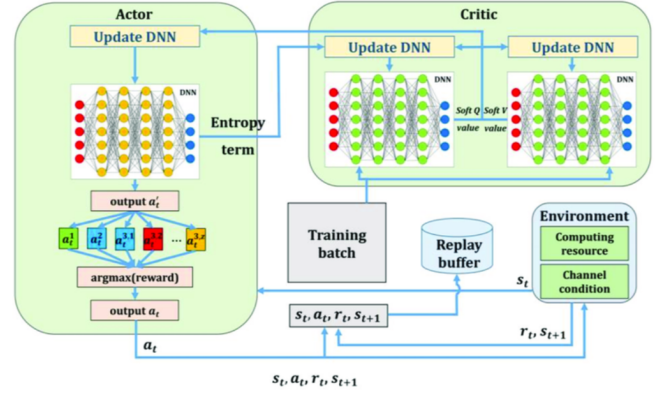


Fig. 11. Soft Actor Critic Architecture

EXPERIMENTAL SETUP

In this section, we will discuss the experimental setup that we used to evaluate the performance of the robot. We used the MuJoCo simulation environment to test the robot under various conditions. In order to validate and accurately assess the effectiveness of the robot, we limited our training to accurately move the robot in a straight line. This was done to ensure that the robot was able to move in a straight line before moving on to more complex terrains.

Trial 1: Torque Control

State $\rightarrow$ POV image of the robot (320x240)

Action $\rightarrow$ Torque values for each joint of the robot

Our initial attempt was to control the robot using torque control. This is to say that the neural network would output the torque values for each joint of the robot. This is a more direct approach to controlling the robot. However, we found that this approach was not effective in controlling the robot on both RL algorithms. The robot would often fall over or not move at all. This was due to the fact that the robot was not able to associate the state (POV image of the robot) with the action (torque values).

Trial 2: Position Control

State $\rightarrow$ POV image of the robot (320x240)

Action $\rightarrow$ Position values for each joint of the robot

We came to realize that there is little to no correlation between the POV image of the robot and the torque values. For this reason, we decided to change the action space to position values for each joint of the robot. This approach was more effective in finding the next position of the robot's joints. However, in order to compute the torques required to move the joint to the desired position, we used the PD controller, which is explained in the next section. Overall these changes did improve the performance of the robot, but it was still not perfect, as the robot would often fall over or not move at all.

PD Controller: The PD controller is a type of feedback controller that uses the error between the desired position and the current position of the joint to compute the torques required to move the joint to the desired position. Every step of the simulation, the PD controller computes the torques required to move the joint to the desired position. The PD controller is defined by the following equation:

$$\begin{aligned} \text{Torque} = & K_p * (\text{Desired_Position} - \\ & \text{Current_Position}) + \\ & K_d * (\text{Desired_Velocity} - \\ & \text{Current_Velocity}) \\ K_p = & 0.5 \\ K_d = & 0.01 \end{aligned}$$

Trial 3: Position Control with Larger State Space

State → POV image of the robot (320x240), joint positions, joint velocities
Action → Position values for each joint of the robot

In order to improve the performance of the robot, we decided to increase the state space of the robot. We added the joint positions and joint velocities to the state space. This allowed the robot to have a better understanding of the environment and the position of the joints. This change did improve the performance of the robot.

Videos of the training can be found in our Google Drive.

NEURAL NETWORK ARCHITECTURE

As discussed above, for all different trials, we used the same architectures for the neural networks, one framework for SAC and one for DQL.

The following neural network is used for both the evaluation and target networks in DQL and the actor and critic networks in SAC.

We did not try different architectures for the neural networks as we found that the current architecture was sufficient for the robot to learn how to move in a straight line.

```
torch.nn.Sequential (
  nn.Linear(76806, 10000),
  nn.LeakyReLU(negative_slope=0.2),
  nn.Linear(10000, 1024),
  nn.LeakyReLU(negative_slope=0.2),
  nn.Linear(1024, action_size))
```

The architecture can be summarized as follows:

- **Input Layer:** The input layer takes the state as input, an image with resolution of 320x240 flattened plus the joint positions and velocities.
- **Hidden Layer 1:** Fully connected layer with ReLU activation.
- **Hidden Layer 2:** Fully connected layer with ReLU activation.
- **Output Layer:** Fully connected layer with the number of actions as output.

RESULTS

Reward Graphs

DQL: Below shows the reward curves of the DQL RL algorithm for trial 1 and trial 3.

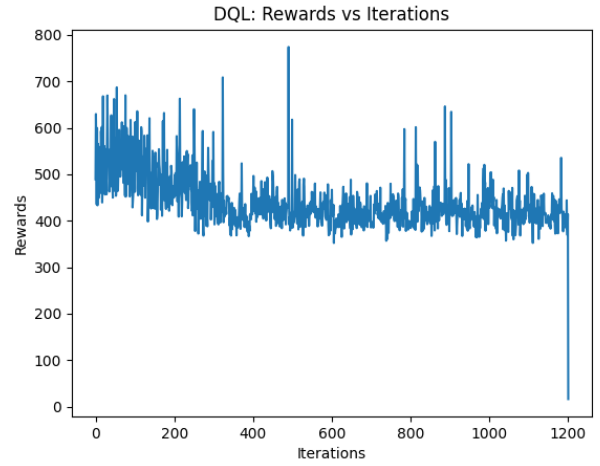


Fig. 12. Reward as the number of iterations increase using the DQL RL algorithm with Torque Control (Trial 1)

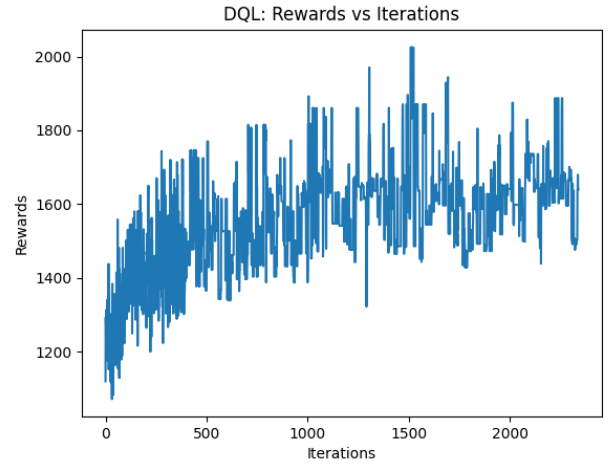


Fig. 13. Reward as the number of iterations increase using the DQL RL algorithm with Position and Velocity Control (Trial 3)

SAC: Below (figures 14 and 15) shows the reward curves of the SAC RL algorithm for trial 1 and trial 3.

Analysis

DQL: Results from the DQL shows that for the initial trial, where torque was the output of the RL policy, there was no improvement in the Rewards per step. In fact, the Reward actually decreased with iterations. However, in trial 3, we incorporated the joint positions and velocities in the state space, which resulted in a significant improvement in the Rewards per step. The robot was able to move in a straight line with a higher reward, which can be seen in figure 13.

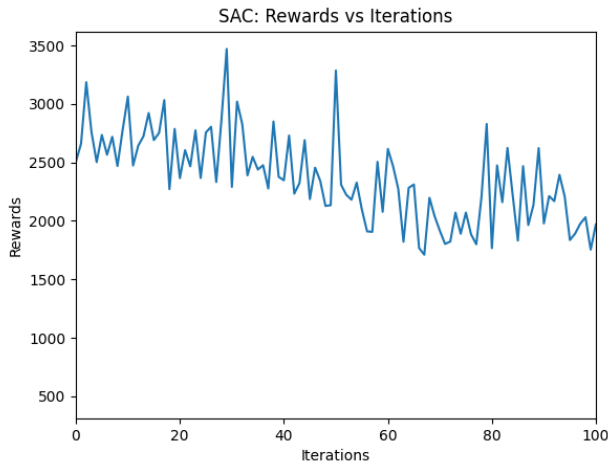


Fig. 14. Reward as the number of iterations increase using the SAC RL algorithm with Torque Control (Trial 1)

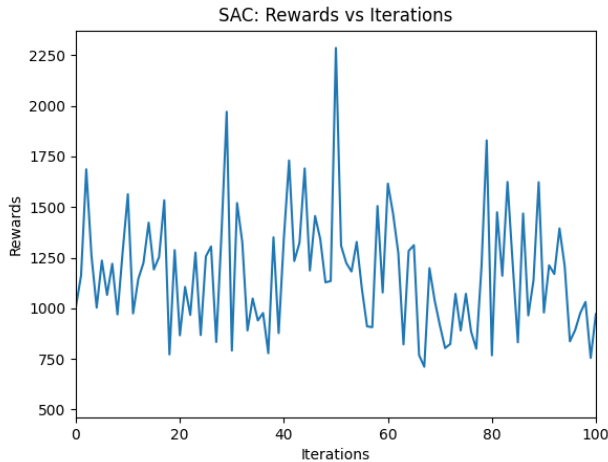


Fig. 15. Reward as the number of iterations increase using the SAC RL algorithm with Joint Position and Velocity Control (Trial 3)

One thing to note is that the magnitude of the reward was drastically different between the two trials, this is because we change the end condition between the two trials. Since each agent's simulation time was longer in trial 3 than trial 1, we see that the average reward time is higher in trial 3.

SAC: Results from the SAC shows that for the initial trial, where torque was the output of the RL policy, there was no improvement in the Rewards per step. In fact, just like DQL, the Reward actually decreased with iterations. However, in trial 3, we incorporated the joint positions and velocities in the state space, which resulted in a slight improvement in the Rewards per step, in the sense that the reward was more stable compared to the initial trial. That being said, the robot was still not able to move in a straight line. Similzr to DQL, the magnitude of the reward was drastically different between the two trials, this is because we change the end condition between the two

trials. Since each agent's simulation time was longer in trial 1 for the SAC algorithm, we see that the average reward time is higher in trial 1.

Additionally, we suspect that the SAC algorithm was not able to learn the optimal policy due to the complexity of the environment or simply due to the lack of training time. We also believe that the SAC algorithm could have performed better by changing the hyper-parameters of the algorithm.

LIMITATIONS

Limitation 1: Training Time

The main limitation of our project is the lack of time to fully train the robot to navigate through complex terrains. We were only able to train the robot to move in a straight line. If time permitted, we would have trained the robot to navigate through complex terrains such as rough terrains, stairs, and slopes (seen in figure 8).

Limitation 2: Real-World Testing

Another limitation of our project is the lack of real-world testing. We were able setup up the robot in the real world, but we were not able to test the robot in the real world mainly due to the lack of the appropriate sensors and hardware. In order to successfully get sim2real transfer, we would need to ensure that the simulation and the real world are as similar as possible. This would require the use of the same sensors and hardware in the real world as in the simulation.

Limitation 3: Simulation Environment

Another limitation of our project is the lack of a realistic simulation environment. We manually created an XML file of the robot in the MuJoCo simulation environment. However, there could have been errors in the XML file that could have affected the dynamics of the robot and effectively the RL policy.

Limitation 4: Lack of Robot URDF File

One of our project goals was to use RL to train PuppyPi, an off the shelf robot dog, to traverse through complex terrains. However, we were not able to obtain the URDF file of the robot. This limited our ability to train the robot in the simulation environment. If we had the URDF file of the robot, we would have been able to train the robot in the simulation environment and then transfer the learned policy to the real world.

CONCLUSION

In this project, BARK9 successfully utilized reinforcement learning, specifically Deep Q-Learning (DQL) and elements of Soft Actor-Critic (SAC), to train a quadruped robot PATRIQ as seen in Fig.1(b) for navigating uneven terrain. The results demonstrated some improvements in the adaptability and stability of PATRIQ in MuJoCo simulations as shown in Fig. 9, allowing it to handle challenging surfaces with greater confidence. These findings highlight the potential of reinforcement learning in developing robust locomotion strategies for quadruped robots in real-world environments.

All of the code and results can be found here:

Github
Google Drive

FUTURE WORK

Future Work 1: Training on Complex Terrains

For future work, we plan to train the robot to navigate through complex terrains such as rough terrains, stairs, and slopes. This would require training the robot in the simulation environment and then transferring the learned policy to the real world. Since we already created the environments in the simulation, the next steps would be to train the robot in the simulation environment and then transfer the learned policy to the real world.

Future Work 2: Dynamic Obstacle Avoidance

Another future work is to train the robot to avoid dynamic obstacles. This would require training the robot in the simulation environment with dynamic obstacles and then transferring the learned policy to the real world. Additionally, we would need to incorporate more sensors such as LIDAR and cameras to detect the dynamic obstacles in simulated environment.

Future Work 3: Sim2Real Transfer

Another future work is to successfully transfer the learned policy from the simulation environment to the real world. This would require to have the same sensors and actuators in the real world as in the simulation environment. This is a huge element of the project as it is critical for a successful real-world deployment.

TABLE I
PROJECT CONTRIBUTIONS

Task	Contributor
Development of Task Logic	Equal Contributions
Preparation of Training Data	Abin George
Simulation Environment Setup	Aditya Rampal
Design of MuJoCo XML File	Aditya Rampal
Implementation of RL Algorithms (DQL, SAC)	Equal Contributions
Neural Network Architecture Design	Equal Contributions
Reward Function Design	Equal Contributions
Experimental Setup for Straight Line Navigation	Equal Contributions
Validation of Results in Simulation	Equal Contributions
Analysis and Interpretation of Reward Graphs	Equal Contributions
Report Writing and Documentation	Equal Contributions
Future Work Suggestions	Equal Contributions

APPENDIX

Figures 16 and 17 are screenshots of the DQL and SAC models being trained on the SCC.

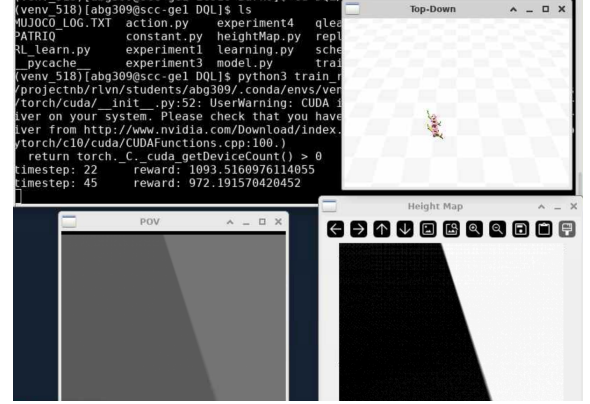


Fig. 16. Screenshot of DQL training being done in the SCC.

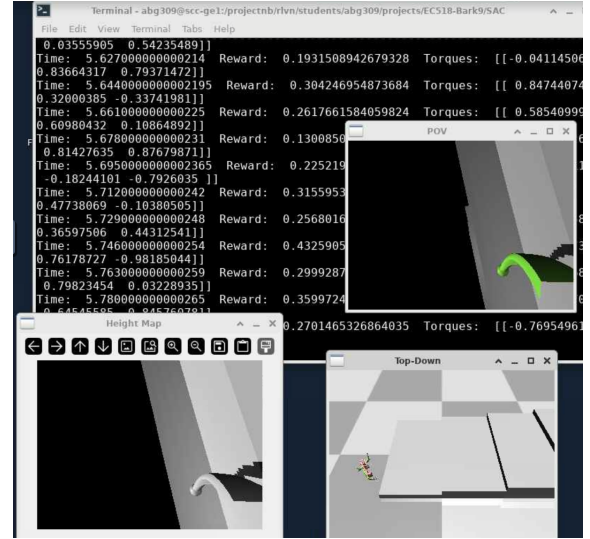


Fig. 17. Screenshot of SAC training being done in the SCC.

REFERENCES

- [1] S. Gangapurwala, M. Geisert, R. Orsolino, M. Fallon, and I. Havoutis, "Real-time trajectory adaptation for quadrupedal locomotion using deep reinforcement learning," in *2021 IEEE International Conference on Robotics and Automation (ICRA)*, 2021, pp. 5973–5979.
- [2] A. W. Winkler, C. D. Bellicoso, M. Hutter, and J. Buchli, "Gait and trajectory optimization for legged systems through phase-based end-effector parameterization," *IEEE Robotics and Automation Letters*, vol. 3, pp. 1560–1567, 2018. [Online]. Available: <https://api.semanticscholar.org/CorpusID:3560219>
- [3] C. Dario Bellicoso, F. Jenelten, P. Fankhauser, C. Gehring, J. Hwangbo, and M. Hutter, "Dynamic locomotion and whole-body control for quadrupedal robots," in *2017 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2017, pp. 3359–3365.
- [4] E. Todorov, T. Erez, and Y. Tassa, "Mujoco: A physics engine for model-based control," in *2012 IEEE/RSJ International Conference on Intelligent Robots and Systems*, 2012, pp. 5026–5033.
- [5] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, "Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor," *CoRR*, vol. abs/1801.01290, 2018. [Online]. Available: <http://arxiv.org/abs/1801.01290>